# Programación declarativa

## • Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

# 19. Operadores

## Definición de nuevos operadores

- Suma de dos listas componente a componente:

```haskell
infixl 6 |+|
(|+|) :: Num a => [a] -> [a] -> [a]
xs |+| ys = map (uncurry (+)) (zip xs ys)
```

- La definición de un operador tiene tres partes:
  - Declaración de asociatividad y prioridad.
    - Es opcional. Por defecto se asume `infixl 9`.
    - `0` es la prioridad más baja; `9` es la más alta.
    - Prioridad más alta = liga más fuerte.
    - `infixl` = en ausencia de paréntesis, se agrupa de izquierda a derecha.
    - `infixr` = en ausencia de paréntesis, se agrupa de derecha a izquierda.
    - `infix` = los paréntesis son obligatorios para agrupar.
  - Declaración del tipo: igual que para funciones.
  - Descripción del comportamiento: igual que para funciones.
- Nombres válidos de operadores:
  - Cualquier secuencia formada por estos caracteres:
    - `! # $ % & * + . / < = > ? @ \ ^ | - ~ :`
    - que no esté ya definida en Haskell.
  - También se permiten muchos símbolos Unicode.
  - No se permiten letras ni números.
  - Los nombres que empiezan por `:` solo pueden usarse como constructores.

## Ejemplos

```haskell
-- producto escalar
infix 7 .*
(.*) :: (Int, Int, Int) -> (Int, Int, Int) -> Int
(x1, x2, x3) .* (y1, y2, y3) = x1*y1 + x2*y2 + x3*y3

-- aritmética módulo 10
infixl 6 +%
infixl 7 *%
(+%), (*%) :: Int -> Int -> Int
x +% y = (x + y) `mod` 10
x *% y = (x * y) `mod` 10

-- doble aplicación
infixr 0 $$
($$) :: (a -> a) -> a -> a
f $$ x = f (f x)
```

## Ejercicios

1. Según las definiciones de los operadores `$$` y `*%` de arriba, ¿cuál es el resultado de evaluar `(\x -> -x) $$ (*%2) $$ 3 - 7`?

2. Define los operadores `+/` y `*/` para sumar y multiplicar, respectivamente, elementos del tipo `Fraccion` de abajo (no es necesario simplificar el resultado). Elige las declaraciones de prioridad y asociatividad que hagan que se comporten, al combinarlos, igual que `+` y `*`: en concreto, `Fraccion 1 2 +/ Fraccion 1 3 */ Fraccion 2 1` se debe interpretar como `Fraccion 1 2 +/ (Fraccion 1 3 */ Fraccion 2 1)`.

```haskell
data Fraccion a = Fraccion a a
```